

Optimizing SGEMM: A High-Performance Approach to Matrix Multiplication

Aaron Ang Jerry Ma
University of California, San Diego

Abstract

We present a high-performance SGEMM (single-precision general matrix multiply) CUDA kernel that combines global memory coalescing, shared-memory tiling, and two-dimensional block tiling. Using 16×16 thread blocks that each compute an 8×8 output tile—a 128×128 tile per block—with an inner K -tile of 32, the kernel exposes instruction-level parallelism through register-blocked outer products and `#pragma unroll`. On an NVIDIA T4 it reaches 4779.9 GFlops at $N = 2048$, approaching cuBLAS, and we analyze how occupancy, register pressure, and arithmetic intensity shape performance—including the degradation that occurs when N is not a multiple of the tile dimension.

1 Development Flow

Our SGEMM implementation achieves efficiency through three key optimization strategies: global memory coalescing, shared memory utilization, and two-dimensional block tiling. Our final implementation is as follows. We employ a thread block dimension of 16×16 (`BLOCKDIM = 16`), resulting in 256 threads per block. Each thread is responsible for computing an 8×8 block (`TILESCALE_M = TILESCALE_N = 8`), which means each thread block processes a 128×128 submatrix of C (`TILEDIM_M = TILEDIM_N = 128`). The tile dimension is derived from `BLOCKDIM  $\times$  TILESCALE = 16  $\times$  8`. We split every `TILEDIM_M` rows of A and `TILEDIM_N` columns of B into sub-block tiles and iterate over them, as shown in Figure 1.

Each sub-block tile has a dimension (`TILEDIM_K`) of 32. We chose this value to remain within our shared memory threshold of 64 KB while enabling efficient memory coalescing patterns. In each iteration over a sub-block tile, all threads in the thread block collaboratively load values from global to shared memory before computing the outer product for the `TILEDIM_M  $\times$  TILEDIM_N` submatrix, as shown in Figure 1.

As shown in Figure 2, each thread performs `TILEDIM_K = 32` matrix outer products. For each iteration over K , individual

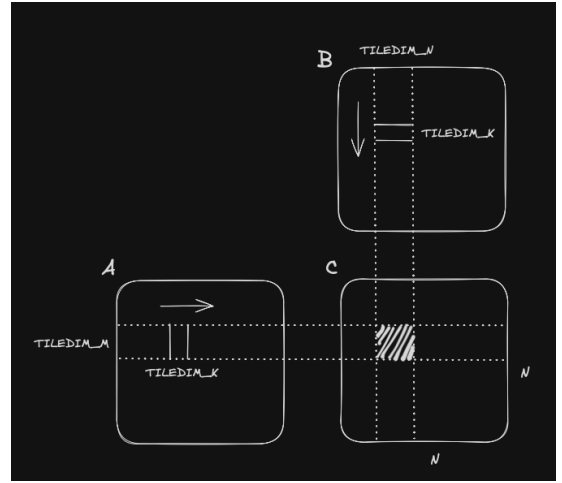


Figure 1: Iterate over sub-block tiles to accumulate the outer product of submatrix C .

threads load 8 values from block tile A and 8 from block tile B into registers. They then execute nested loops to accumulate the 8×8 result locally before writing the final results back to matrix C in global memory. Computing the outer product exposes substantial instruction-level parallelism (ILP) since there are no dependencies between instructions here. Paired with the `#pragma unroll` directive, we can guide the compiler to perform loop unrolling to achieve this ILP while keeping the code neat and flexible across various configurations.

To handle edge cases where matrix dimensions aren't multiples of 128, we implement bounds checking. During shared memory loading, we calculate global row and column positions as the product of thread block coordinates and `TILEDIM`, validating these bounds before transferring data from global to shared memory. This prevents result corruption from out-of-bounds data. Similarly, when writing results, we check bounds to ensure memory safety at matrix edges. The code is shown in Figures 3 and 4.

Our development progressed through several stages. We

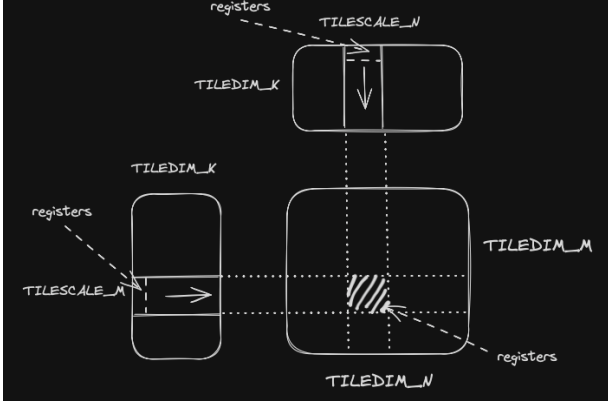


Figure 2: Each thread calculates a $\text{TILESCALE_M} \times \text{TILESCALE_N}$ outer product.

```
// populate the SMEM caches
#pragma unroll
for (uint loadOffset = 0; loadOffset < TILEDIM_M; loadOffset +=
    strideA) {
    const uint globalRowA = globalRowOffset + innerRowA +
        loadOffset;
    const uint globalColA = bkIdx + innerColA;
    As[innerRowA + loadOffset][innerColA] =
        (globalRowA < N && globalColA < N)
        ? A[(innerRowA + loadOffset) * N + innerColA]
        : 0.0f;
}
#pragma unroll
for (uint loadOffset = 0; loadOffset < TILEDIM_K; loadOffset +=
    strideB) {
    const uint globalRowB = bkIdx + innerRowB + loadOffset;
    const uint globalColB = globalColOffset + innerColB;
    Bs[innerRowB + loadOffset][innerColB] =
        (globalRowB < N && globalColB < N)
        ? B[(innerRowB + loadOffset) * N + innerColB]
        : 0.0f;
}
__syncthreads();
```

Figure 3: Bounds checking when populating shared memory.

started with shared memory optimization, which improved on our coalesced global-memory kernel by about 20% at small N and up to about 70% at $N = 2048$ (Figure 8). We then implemented 1-D block tiling where each thread calculated a column of results, nearly doubling performance. The transition to two-dimensional block tiling was the hardest as we misunderstood the definitions of blocks and tiles. We tried various configurations but performance dropped significantly below that of 1-D block tiling. We believe it is due to bad coalesced loads when loading to shared memory and performing too many computations per thread. Thankfully, we clarified with TA Thomas during office hours that a block tile should represent the values computed by an entire thread block. With this insight and guidance from CUTLASS [3] and Simon Boehm’s blog [5], we successfully implemented our final solution.

```
// write out the results
#pragma unroll
for (uint resIdxM = 0; resIdxM < TILESCALE_M; ++resIdxM) {
    #pragma unroll
    for (uint resIdxN = 0; resIdxN < TILESCALE_N; ++resIdxN) {
        const uint tileRow = threadRow * TILESCALE_M + resIdxM;
        const uint tileCol = threadCol * TILESCALE_N + resIdxN;
        if (globalRowOffset + tileRow < N &&
            globalColOffset + tileCol < N) {
            C[tileRow * N + tileCol] =
                threadResults[resIdxM][resIdxN];
        }
    }
}
```

Figure 4: Bounds checking when writing results back to C .

2 Results

Throughout this section, we use the shorthand T_M , T_N , and T_K for TILEDIM_M , TILEDIM_N , and TILEDIM_K ; B_x and B_y for the block dimensions BLOCKDIM_X and BLOCKDIM_Y ; and S_N for the tile scale TILESCALE_N .

(a) Configuration and access-pattern constraints. We fix $T_K = 32$, which means for each block tile, we divide N into N/T_K subtiles of A of size $T_M \times T_K$, and in each iteration, we perform the outer product $A_{T_M:N} \otimes B_{N:T_N}$.

The thread block and tile sizes are limited by the access pattern of A when loading into shared memory:

$$B_x \times B_y \geq T_K.$$

Similarly for B :

$$B_x \times B_y \geq T_N = S_N \times B_y.$$

The above equations can be simplified to:

$$B_x \geq S_N.$$

(b) Occupancy analysis. Based on Figure 5, the optimal thread block size is 16×16 across all N . However, we observed some irregular performance behaviors when $N = 256$. We analyze the potential reasons for the performance differences from the perspective of the number of warps and the possible maximum occupancy, as summarized in Table 1.

One intuitive explanation for why the configurations $\text{bx}=32, \text{by}=32, \text{tm}=2, \text{tn}=2$ and $\text{bx}=16, \text{by}=16, \text{tm}=4, \text{tn}=4$ perform better at $N = 256$ is that they require more warps, which leads to more concurrent SM usage. With 128×128 tiles, $N = 256$ yields only 4 thread blocks, so 36 of the T4’s 40 SMs sit idle.

The second reason is the per-SM resource limit. For $\text{bx}=16, \text{by}=16, \text{tm}=8, \text{tn}=8$, a hand count gives $8 + 8 + 8 \times 8 = 80$ registers per thread, but that ignores pointers, indices, and loop state; ptxas reports 172. The T4 has 64K 32-bit registers, at most 32 resident warps, and 64 KB of shared memory per SM. A 256-thread block (8 warps) at 172 registers needs

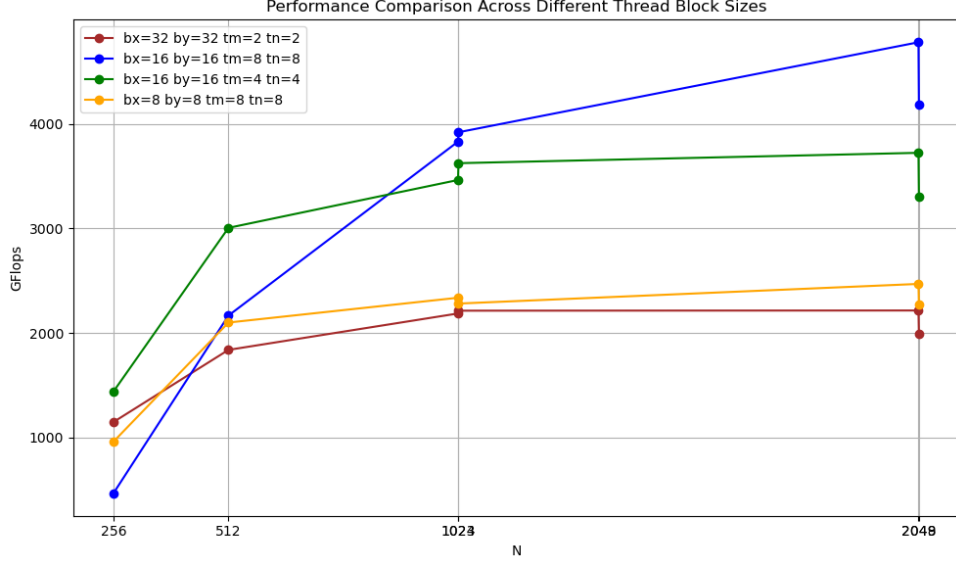


Figure 5: Performance across various block dimensions and tile scales.

Table 1: Occupancy analysis for several thread-block and tile configurations at $N = 256$ on the NVIDIA T4. Regs/Thr. is the `ptxas` count for `sm_75` (CUDA 13); Warps is the total over the grid. Occupancy is shown with the original 64 KB dynamic shared-memory request and with the exact tile size.

Configuration	Tile Dim.	Blocks	Thr./Block	Threads	Regs/Thr.	Warps	Occ. (64 KB)	Occ. (exact)
bx=32 by=32 tm=2 tn=2	64	16	1024	16384	64	512	100%	100%
bx=16 by=16 tm=8 tn=8	128	4	256	1024	172	32	25%	25%
bx=8 by=8 tm=8 tn=8	64	16	64	1024	222	32	6.25%	25%
bx=16 by=16 tm=4 tn=4	64	16	256	4096	91	128	25%	50%

about 45K registers (allocated in 256-register units per warp), so only $\lfloor 65536/45056 \rfloor = 1$ block fits per SM. Even at the hand-counted 80 registers (2560 per warp), registers would allow $\lfloor 65536/(8 \times 2560) \rfloor = 3$ blocks, i.e., 24 warps (75%).

Shared memory adds a second cap. The kernel uses only $(T_M T_K + T_K T_N) \times 4B = 32KB$ for A_S and B_S , but our host code requested 64 KB of dynamic shared memory per block, which by itself allows only one block per SM. Both limits give $8/32 = 25\%$ occupancy. We have since changed the request to the exact tile size. Shared memory then admits two blocks (50%), but at 172 registers the register file still caps this configuration at one block. Reaching 50% would need at most 128 registers per thread (e.g., `__launch_bounds__(256, 2)`). The fix does raise the smaller-tile configurations in Table 1, which were held to one block per SM by the 64 KB request.

We noticed that high occupancy is not always needed to reach high performance when the arithmetic intensity q is small or large. q is given by:

$$q = \frac{f}{m},$$

where f is the number of floating point operations, which is $2N^3$, and m is the number of 4-byte elements moved to

or from global memory. Each block tile loads $N(T_M + T_N)$ values from matrices A and B into shared memory A_S and B_S , and the number of block tiles is $N^2/(T_M T_N)$. We also perform N^2 store operations back to global memory. Thus, in flops per element,

$$q = \frac{2N^3}{\frac{N^3(T_M + T_N)}{T_M T_N} + N^2} = \frac{2N T_M T_N}{N(T_M + T_N) + T_M T_N}.$$

If we consider the square cases where $T_M = T_N$, we can simplify q as:

$$q = \frac{2N T_M}{2N + T_M} \text{ flop/element} = \frac{N T_M}{2(2N + T_M)} \text{ flop/byte},$$

which approaches $T_M/4 = 32$ flop/byte for $T_M = 128$ (31.0 at $N = 2048$), past the T4’s ridge point of $7680/320 = 24$ flop/byte. When T_M is large, we become compute-bound and higher occupancy would not lead to better throughput, as explained in the cusp behavior described by Volkov [2]. For the configuration `bx=16, by=16, tm=8, tn=8`, it has the highest T_M of 128, yet it outperforms other configurations despite its 25% occupancy: its per-thread ILP (64 independent FMAs

Table 2: Peak GFlops and the corresponding thread block size for each matrix size.

N	Peak GF	Thread Block Size
256	1148.9	bx=16 by=16 tm=4 tn=4
512	2166.9	bx=16 by=16 tm=8 tn=8
1023	3829.8	bx=16 by=16 tm=8 tn=8
1024	3919.6	bx=16 by=16 tm=8 tn=8
2048	4779.9	bx=16 by=16 tm=8 tn=8
2049	4181.0	bx=16 by=16 tm=8 tn=8

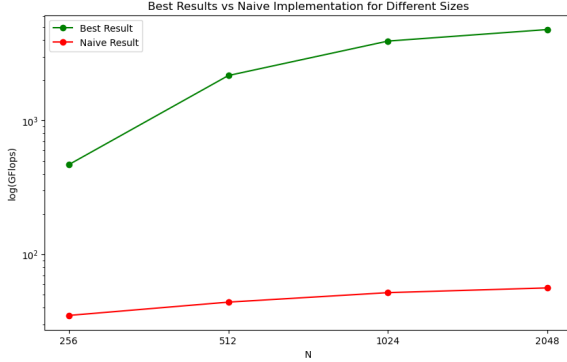


Figure 6: log(GFlops) comparison with the naive implementation.

per k step) hides latency that other configurations need more resident warps to cover.

(c) Peak performance by matrix size. Table 2 documents the peak GFlops achieved and the corresponding thread block size for each matrix size.

3 Comparison to Naive

Figure 6 compares the throughput of our kernel against the naive implementation on a logarithmic scale, and Figure 7 shows the corresponding speedup.

4 Analysis

(a) Performance across implementations. Figure 8 shows performance across our different implementations.

(b) Resemblance to cuBLAS. We noticed that the performance curve of our 2D-tiling-64-outputs method closely resembles that of cuBLAS, with both showing fluctuations before converging to around 4000+ GFlops. One possible explanation for this similarity is that both methods are based on the block-tile approach.

(c) Performance anomalies. We noticed two unusual behaviors in performance.

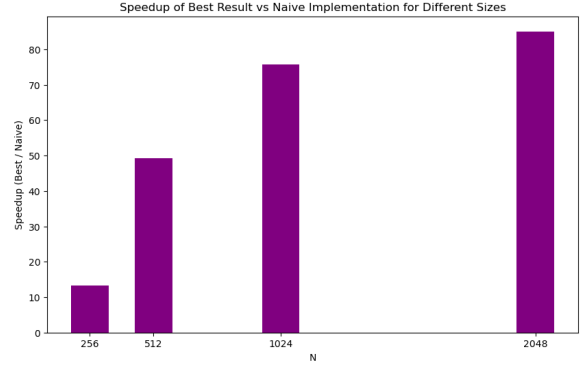


Figure 7: Speedup over the naive implementation.

1) *Performance degradation when N is not a multiple of 128.* Our 2D-tiling-64-outputs implementation and cuBLAS exhibit a significant performance drop when the matrix size N is not a multiple of 128. In contrast, other implementations, such as 1-D blocking and naive methods, show much less fluctuation. The main cause is wave quantization. Our kernel fits one block per SM (Table 1), and the T4 has 40 SMs, so a launch of $\lceil N/128 \rceil^2$ blocks runs in $\lceil \lceil N/128 \rceil^2 / 40 \rceil$ waves, and a partial last wave leaves SMs idle. $N = 1023$ and 1024 both launch 64 blocks (2 waves), while $N = 1025$ launches 81 blocks and needs a third wave for a single block. That predicts roughly $2/3$ of the 3920 GFlops at $N = 1024$, about 2600 GFlops. We measured about 2900, so the model is somewhat pessimistic but accounts for most of the drop. Likewise, $N = 2048$ launches 256 blocks (7 waves), and $N = 2049$ launches 289 (8 waves). The prediction, $4780 \times 7/8 \approx 4183$ GFlops, matches the measured 4181.0. $N = 1199$, 1200, and 1201 all launch 100 blocks (3 waves), which explains why these three sizes perform almost the same.

Control divergence adds a smaller cost. Boundary checks that zero-fill As and Bs send threads in a warp down different paths, and edge tiles do wasted work. This explains the small gap between $N = 1023$ (3829.8 GFlops) and $N = 1024$ (3919.6), but not the much larger gap between 1023 and 1025.

2) *Performance degradation when N is greater than 768.* When $N > 768$, we observe that performance no longer improves significantly and, in some cases, even degrades for certain matrix sizes. The kernel is not memory-bound here: with a fixed 8×8 block per thread, arithmetic intensity plateaus near $T_M/4 = 32$ flop/byte, above the T4's ridge point of 24. Counting the padded tiles ($\lceil N/T_M \rceil^2$ blocks, each loading $2NT_M$ elements, plus N^2 stores of 4 B each) gives the sawtooth in Figure 9:

$$q = \frac{2N^3}{4(\lceil N/T_M \rceil^2 \cdot 2NT_M + N^2)} \text{ flop/byte.}$$

The plateau instead reflects wave quantization and the compute ceiling. $N = 768$ launches 36 blocks, one nearly full wave, and reaches about 4150 GFlops, while $N = 900$

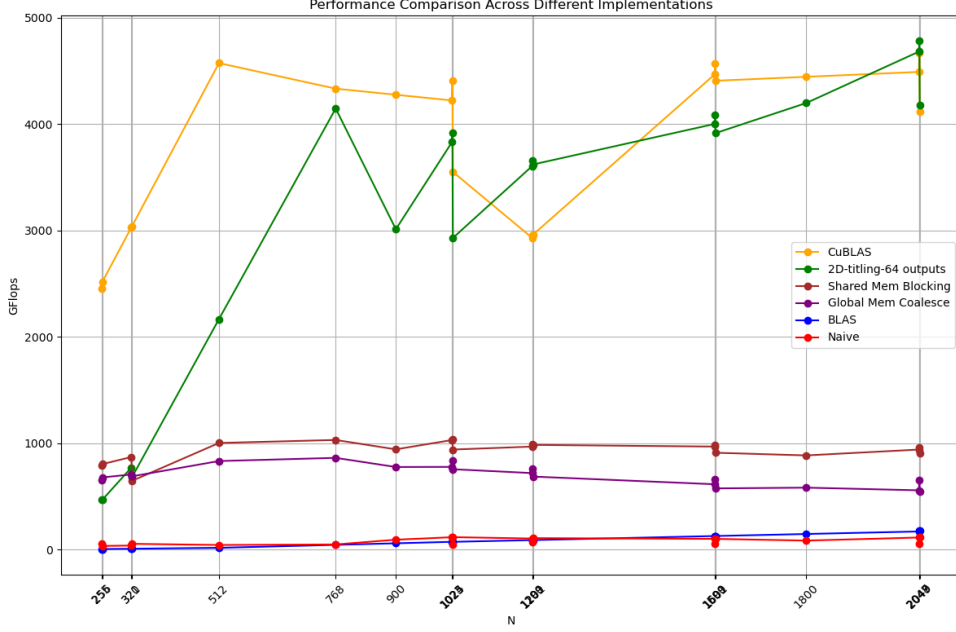


Figure 8: Performance across different implementations.

launches 64 blocks, whose second wave fills only 24 of 40 SMs, and drops to about 3000. As N grows, the partial last wave matters less and throughput approaches the 4780 GFlops seen at $N = 2048$.

5 Calculations

(a) Peak performance and arithmetic intensity. Peak performance is the total operations per second that a GPU can perform, assuming all its cores execute their maximum number of ops per cycle, where frequency is cycles per second per core. Thus,

$$\begin{aligned}
 \text{peak} &= \text{SMs} \times \frac{\text{cores}}{\text{SM}} \times \text{freq} \times \frac{\text{ops}}{\text{cycle}} \\
 &= 40 \times 64 \times (1.5 \times 10^9) \times 2 \\
 &= 7680 \text{ GFlops/s.}
 \end{aligned}$$

For $n = 2048$, we achieved 4779.9 GFlops. To calculate the arithmetic intensity q , we divide the number of flops ($2n^3$) by the bytes moved ($2n^3/\text{TILEDIM_M}$ element loads and n^2 element stores, 4 B each), giving $q \approx 31$ flop/byte, past the ridge point, as shown in Figure 10.

(b) Effect of reduced memory bandwidth. The ridge point is $q^* = \text{peak flops/bandwidth} = 7680/320 = 24.0$ flop/byte. At the measured 220 GB/s, it rises to $q^* = 7680/220 = 34.9$ flop/byte. A kernel now needs higher arithmetic intensity to reach peak. At about 31 flop/byte, ours would fall just below the new ridge, but its bandwidth bound ($220 \times 31 \approx$

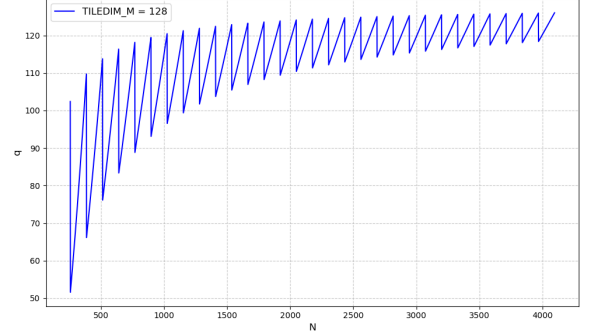


Figure 9: Arithmetic intensity of our kernel over N , plotted in flop per 4-byte element (divide by 4 for flop/byte).

6800 GFlops) is still above the 4780 GFlops we achieve (Figure 11).

6 Potential Future Work

Given more time, we would have explored interleaved computation as a potential optimization. Since our current implementation already achieved strong performance, we decided not to pursue it. This technique would have required a substantial departure from our outer-product approach, which focuses on computing contiguous 2-D blocks. The interleaved method would mean redesigning our thread block dimensions and tile sizes, effectively requiring an architectural rethink rather than an incremental improvement to our existing design.

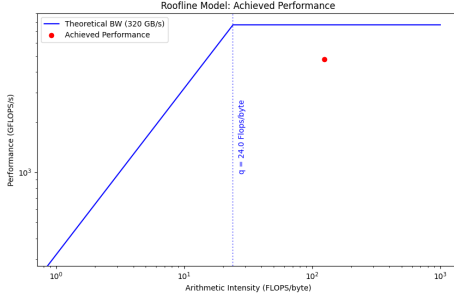


Figure 10: Roofline model: achieved performance at $N = 2048$. The dotted line marks the ridge point (24.0 flop/byte), not the kernel’s intensity. The plotted point uses flop per element (≈ 124), i.e., ≈ 31 flop/byte.

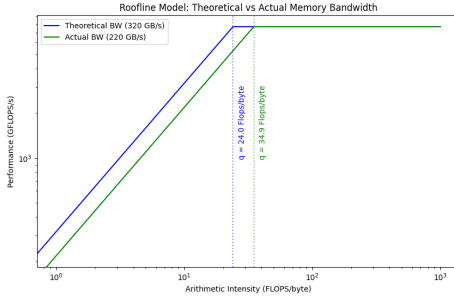


Figure 11: Roofline model: theoretical versus actual memory bandwidth (ridge points $q^* = 24.0$ versus 34.9 flop/byte).

7 Extra Credit

We created a non-square matrix multiplication kernel with the following header in `mmpy_kernel.cu`:

```
__global__ void matMulNonSquare(
    int M, int N, int K,
    _FTYPE_ *__restrict__ C,
    _FTYPE_ *__restrict__ A,
    _FTYPE_ *__restrict__ B)
```

This required three changes to our original implementation. First, we updated the outer loop bound from N to K , since K represents the shared dimension between matrices A and B used to iterate the sub-block tiles (refer to Figure 1). Second, we modified the shared memory loading bounds to check against M for matrix A and N for matrix B . Finally, when writing results back to matrix C , we updated our bounds checking to compare the global row offset against M and the global column offset against N , since the output is now an $M \times N$ matrix.

To optimize our non-square matrix multiplication kernel, we could implement several changes. Instead of using fixed 128×128 tile sizes, we could dynamically adjust tile dimensions M , N , and K . For example, if M is much larger than N , we can scale `TILEDIM_M` by a larger factor (using

`TILESCALE_M`) up to the shared memory limit. Essentially, our outer product dimensions would be proportional to M and N , maximizing the amount of work done per thread. When K is much larger than M or N , we could increase `TILEDIM_K` (while adjusting other parameters) to maximize shared memory utilization. These changes would enable the matrix multiplication computation to remain highly performant across varying input sizes.

References

- [1] V. Volkov. *Understanding Latency Hiding on GPUs*. Ph.D. thesis, University of California, Berkeley, Technical Report UCB/EECS-2016-143, 2016. <https://www2.eecs.berkeley.edu/Pubs/TechRpts/2016/EECS-2016-143.html>.
- [2] V. Volkov. Better performance at lower occupancy. In *GPU Technology Conference (GTC)*, 2010. https://www.nvidia.com/content/gtc-2010/pdfs/2238_gtc2010.pdf.
- [3] NVIDIA. CUTLASS: Fast linear algebra in CUDA C++. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/cutlass-linear-algebra-cuda/>.
- [4] Must know technique in GPU computing: Tiled matrix multiplication in CUDA C. <https://www.youtube.com/watch?v=Q3GgbfGTnVc>.
- [5] S. Boehm. How to optimize a CUDA matmul kernel for cuBLAS-like performance: A worklog. <https://siboehm.com/articles/22/CUDA-MMM>.